Università di Genova

Embedded Systems

On-going Assessment

Prof. Enrico Simetti

On-going Assessment

1/ Simulate an algorithm that needs 7 ms for its execution and needs to work at 100 Hz. This is to emulate a real-world scenario.

2/ Make LD2 blink at 1 Hz (500 on, 500 off).

3/ Read characters from UART1. If a string $BW,xx* is received, the xx dictates the bandwidth of the accelerometer filter. Valid data for xx are 8 to 15. Invalid values (e.g., -1, 3.2, 18) should be discarded and a message $ERR,1* should be sent on the UART. The initial bandwidth is 1000 Hz (value 15, see page 58). If a string $HZ,yy* is received, the yy dictates the frequency of the data sent over the UART. Valid data for xx are 0, 1, 2, 5, 10. A value of 0 disables the message. Invalid values should be discarded and a message $ERR,1* should be sent on the UART. The initial frequency is 10 Hz.

4/ Acquire the three accelerometer axes at 50 Hz.

5/ Compute the roll and pitch angles

6/ Send it to the UART at yy Hz using the protocol $ACC,x,y,z*.

7/ Send the computed angles at 5 Hz using the message $ANG,x,y*, where x is the roll angle in degrees and y is the pitch angle in degrees.

Interrupts & Hardware Interaction – Key Guidelines

- **Use interrupts only when they are truly needed**
  Before introducing an interrupt, consider whether a simpler approach (e.g., polling or restructuring the code) would be sufficient and easier to reason about
- **Handle shared variables with care**
  If an interrupt and the main program access the same data, ask yourself: *what happens if an interrupt occurs at this exact point?* If it can lead to inconsistent or partial updates, temporarily disable interrupts and re-enable them afterward
- **Consider that interrupts can occur at any time**
  Even in the middle of a single C++ statement, execution may be interrupted—design your code with this in mind
- **Be cautious with busy-waiting on hardware**
  Busy-waiting can block the CPU and delay other tasks; ensure that it will not cause missed deadlines or responsiveness issues. Use interrupts instead

Suggestions

- The availability of only 3 ms to perform operations suggests the use of interrupts to handle the UART, both in reception and transmission.
- The suggested way is to use separate circular buffers to store data coming and going to the UART.
- Given that the IMU allows an SPI clock up to 7.5 MHz, the SPI interactions can be consider negligible (two bytes require around 2 microseconds at 7.5 MHz).
- Do you have any warning when compiling? Solve them! Sometimes, a warning hides a nasty bug (e.g., truncation due to overflow.)

Evaluation Criteria

- Are the tasks executed at the correct frequency? Are there missed deadlines because of some erroneous handling of the peripherals? Are busy-waiting performed when allowed?
- Is the shared data handled correctly? Is there any possibility of wrong sequence of interrupts that might cause some erroneous behaviour?
- Are circular buffers sized correctly? i.e., have you thought on how fast data is produced/received? Memory is a scarce resource in embedded systems.
- Are the interrupts short in time? Are you performing unnecessary busy-waiting inside interrupts? Note: not all while loops are busy-waiting loops.
- Is the code sufficiently commented, well written, formatted? Maintainability and quality of the code is key in embedded applications.

Final Notes

- In case of doubt, ask! Interactions are not forbidden, they are recommended.
- Only a single group participant should submit the final project on Aulaweb. Please submit the whole project folder.
- Use your group ID when creating the project in MPLAB X.

Template Code

```
void algorithm() {
  tmr_wait_ms(TIMER2, 7);
}
int main() {
  tmr_setup_period(TIMER1, 10);
  while(1) {
    algorithm();
    // code to handle the assignment
    ret = tmr_wait_period(TIMER1);
  }
}
```



Università
di Genova